

哈希表 (Hashing) — 考前速查

考点

核心突破：不通过比较查找， $O(1)$ 定位。
基于比较的查找下界 $\Omega(\log N)$ ，哈希突破了它。

1. 基本概念

b (buckets)	桶数量
s (slots/bucket)	每桶槽位数
n	当前元素数
T	所有可能 key 总数
$\lambda = \frac{n}{s \cdot b}$	负载密度

```
Node* p = table[key % T];
while (p && p->key != key) p = p->next;
return p;
}
```

缺点：需要额外指针空间，缓存不友好（内存不连续）。

冲突 (Collision)： $f(i_1) = f(i_2)$ ，两不同 key 映射到同桶。

溢出 (Overflow)：桶已满又来了新 key。

当 $s = 1$ 时，冲突 = 溢出。

例子：10 个 C 函数， $b=26, s=2, \lambda = 10/52 = 0.19$ 。

$\text{hash}(x)$ = 首字母-'a'。acos/atan 都在桶 0。平均查找 = 3.73。

2. 哈希函数

要求：计算快 + 分布均匀（无偏）。

整数：

```
h(x) = x % TableSize
```

TableSize 必须质数！ 否则后几位相同者扎堆。

字符串—Hash3 (推荐)：

```
int hash(const char* key, int T) {
    unsigned int h = 0;
    while (*key) h=(h<<5)+*key++; // h*32+下一字符
    return h % T;
}
```

用 32 因为 $32 = 2^5$ ，移位比 $*27$ 快。

长字符串：可精选部分字符，早期字符被左移出去丢失影响。

3. 分离链接法

每桶挂一个链表。冲突的串在一起。

```
typedef struct Node { int key; struct Node* next; } Node;
Node* table[T];

void insert(int key) {
    int h = key % T;
    Node* node = malloc(sizeof(Node));
    node->key = key;
    node->next = table[h]; // 头插法 O(1)
    table[h] = node;
}

Node* find(int key) {
```

4. 开放地址法

冲突后按探测序列找下一个空位：
 $h_i = (\text{hash}(\text{key}) + f(i)) \bmod \text{TableSize}$

线性探测 $f(i) = i$

每次 +1。一次聚集：hash 相同的人扎堆。
期望探测次数：插入失败 $\frac{1}{2}(1 + 1/(1 - \lambda)^2)$

二次探测 $f(i) = i^2$

+1,+4,+9,+16...
定理：若 TableSize 是质数且表 > 1/2 空，一定找到空位。
(前 T/2 个 $i^2 \bmod$ 质数互不相同)
二次聚集：同 hash 者探测路径一样。

代码：

```
int find(int T[], int key, int size) {
    int h = key % size;
    for (int i=0; i<=size/2; i++) {
        int p = (h + i*i) % size;
        if (T[p]==EMPTY) return -1;
        if (T[p]==key) return p;
    }
    return -1;
}
```

插入优化公式： $f(i) = f(i - 1) + 2i - 1$ (避免每次算 i^2)。
懒惰删除：加 flag 标记删除，不真正清空。
双哈希 $f(i) = i \cdot h_2(\text{key})$
推荐： $h_2(x) = R - (x \% R)$, R 为小于 TableSize 的质数 (如 7)。
步长取决于第二个哈希函数，基本消除聚集。

三种探测对比

	聚集	性能
线性	一次聚集严重	最差
二次	二次聚集	中等
双哈希	基本无	最好

5. 再哈希 (Rehashing)

何时触发：

- 插入失败 (没空位)
- λ 超阈值 (如 > 0.5)
- 查找性能严重下降

步骤：

1. 建更大的表 (约 2 倍，质数)
2. 旧表全部元素重哈希插入新表
3. 释放旧表

$O(N)$ 一次，但摊还到每次插入 $O(1)$ 。
交互系统注意：不能一次性停太久——可逐步迁移。

6. 分离链接 vs 开放地址

	分离链接	开放地址
实现	链表	纯数组
空间	额外指针	无额外开销
满后	无上限	Rehashing
λ	可 >1	必须 <1
缓存	差	好 (连续)

7. 线性探测反推输入顺序

考点

给定哈希表状态，反推插入顺序。线性探测下，元素从 hash 到实际位置之间经过的所有人都比它先插入。

入度公式： $\text{indeg}[i] = (i - \text{hash}(a[i]) + N) \% N$

然后拓扑排序：每次选入度为 0 且值最小的输出。

例子： N=5, 表 =[10,6,15,0,8]。
a[4]=8,h=3,in=1(依赖 3); a[3]=0,h=0,in=3(依赖 0,1,2)...
最小候选优先输出。